AI Systems Performance Engineering

Chapter 6 — GPU Architecture, CUDA Programming, and Maximizing Occupancy

Your Name
Applied Scientist
AWS

Internal Training for Applied Scientists on AI Infrastructure

August 15, 2026

What This Talk Is About — and Why

In plain English

A GPU is a **throughput machine**: it runs **thousands of threads** at once and hides slow memory by always having other work ready. This chapter teaches the **vocabulary** (warps, blocks, grids, SMs), the **memory ladder** (registers → shared → L2 → HBM), and the **one golden rule**: launch enough parallel work to keep every SM busy.

- **The problem:** a GPU with idle SMs is an expensive space heater — most naive kernels use a few percent of the hardware.
- **The tool:** CUDA's thread hierarchy maps your algorithm onto hardware, unchanged across GPU generations.
- **The metric:** *occupancy* — how many warps are in flight vs. the hardware maximum.
- **The compass:** the *roofline model* tells you if a kernel is compute-bound or memory-bound before you optimize the wrong thing.

This is the foundation chapter for everything kernel-level in Chapters 7–12.

Roadmap

1. **GPU architecture** — SMs, warp schedulers, threads/blocks/grids, warp divergence
2. **CUDA programming refresher** — kernel anatomy, launch parameters, async allocation
3. **Memory hierarchy** — registers, shared/L1, TMEM, L2, HBM3e, Unified Memory
4. **Occupancy** — why parallelism is rule #1; a 22× case study; launch bounds
5. **Correctness** — NVIDIA Compute Sanitizer
6. **Roofline model** — compute-bound vs. memory-bound, and what to do about each

Plain English: first make the GPU busy, then make each cycle count, and always profile to know which fix applies.

GPUs Optimize Throughput; CPUs Optimize Latency

- CPUs: few cores, deep caches, fast *single* threads. GPUs: hundreds of **SMs** (streaming multiprocessors) running **thousands of threads** in parallel.
- Basic CUDA flow: host copies data **CPU → GPU**, launches a **kernel**, copies results back.
- The GPU hides transfer and memory latency with **massive parallelism**: when one warp waits, another runs.
- Each Blackwell SM tracks up to **64 warps** (2,048 threads) concurrently.[1,2]

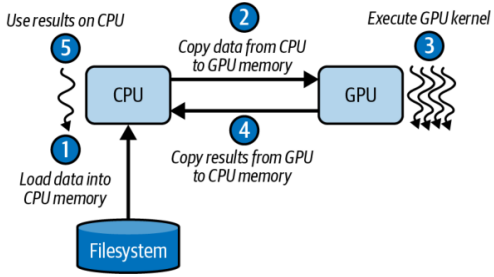

Figure 6-1. Simple CUDA programming flow

Plain English: the CPU is a sports car, the GPU is a freight train — don't ask the train to be fast, ask it to be full.

Inside a Blackwell SM: Four Schedulers, Dual Issue

- Each SM = four "**mini-SMs**": independent warp schedulers sharing on-chip resources.
- Each scheduler issues **1 warp instruction/cycle**, and can **dual-issue** one math + one memory op *from the same warp*.
- Best case: **4 math + 4 memory** instructions per cycle across 4 warps.
- **SFUs** (sin, cos, sqrt, ...) run in a separate pipeline — transcendentals don't stall the core pipes.
- Per SM: **64K 32-bit registers** (256 KB), **256 KB** unified L1/shared memory.[2]

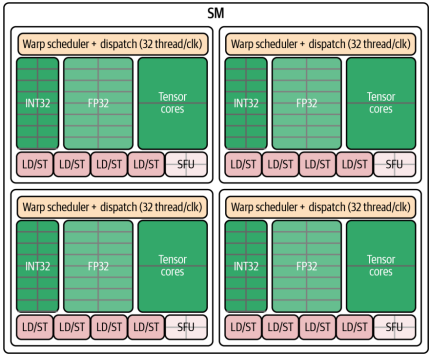

Figure 6-2. Four warp schedulers with dual issue (math + memory)

Plain English: four checkout lanes; each cashier scans with one hand and bags with the other.

The Thread Hierarchy: Threads → Blocks → Grids

- **Thread**: runs your kernel code on one data element.
- **Thread block** (CTA): up to **1,024 threads**; shares fast **shared memory**; synchronizes with `__syncthreads()` (minimize barriers — they cost time).
- **Grid**: all blocks of one launch; scales to **millions of threads** with no kernel changes.
- Blocks execute **independently, in any order** — that freedom is what lets the same code run on future GPUs with more SMs.

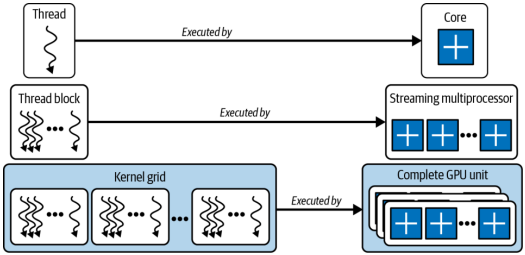

Figure 6-3. Threads, thread blocks (CTAs), and grids

Plain English: workers (threads) form teams (blocks) that can talk cheaply inside the team; the company (grid) hires as many teams as the job needs.

Warps and SIMT: 32 Threads in Lockstep

- Blocks are subdivided into **warps of 32 threads** that execute one instruction together (**SIMT**: single instruction, multiple threads).
- **Occupancy** = fraction of an SM's warp slots that are occupied by active warps. High occupancy ⇒ when one warp stalls on memory, another is ready ⇒ **latency hiding**.
- Balance against per-thread resources: too many registers/shared memory per thread ⇒ fewer resident warps, or **register spilling** to slow memory.

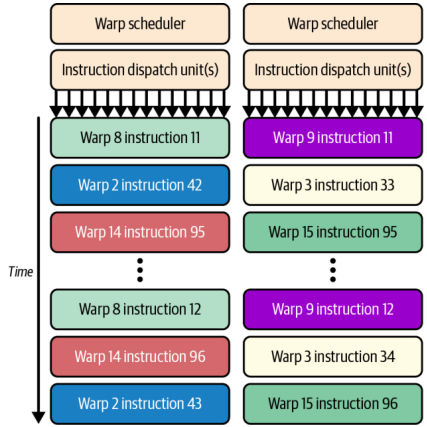

Figure 6-7. A warp advances as a whole under the warp scheduler

Plain English: a warp is a rowing crew of 32 — everyone pulls the same stroke at the same time, or the boat waits.

Warp Divergence: When `if/else` Splits the Crew

- If threads *within one warp* take different branches, the warp **serializes**: it runs the `if` path with half the lanes masked, then the `else` path.
- Execution time multiplies by the **number of distinct paths**.
- **No penalty across warps** — different warps may branch differently for free.
- Detection and mitigation: Chapter 8 (profile with Nsight Compute).

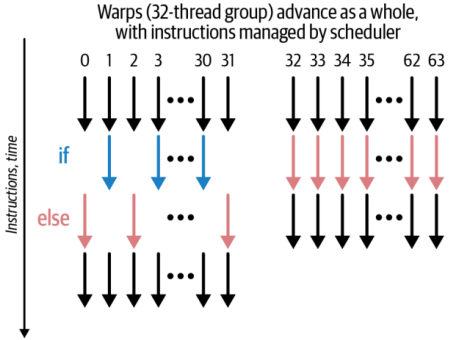

Figure 6-8. SIMT warp divergence (left) vs. uniformity (right)

Plain English: if half the rowing crew rows left and half rows right, the boat does each maneuver twice — slowly.

Hardware Limits That Shape Your Launch (Blackwell B200)

Resource	Limit
Warp size	32 threads
Max threads / block	1,024 (= 32 warps)
Max resident warps / SM	**64** (2,048 threads)
Max resident blocks / SM	32
Registers / SM	64K × 32-bit (255 / thread)
Shared memory / SM	228 KB (227 usable)
Max grid (X dim)	2,147,483,647 blocks
Max concurrent grids	128

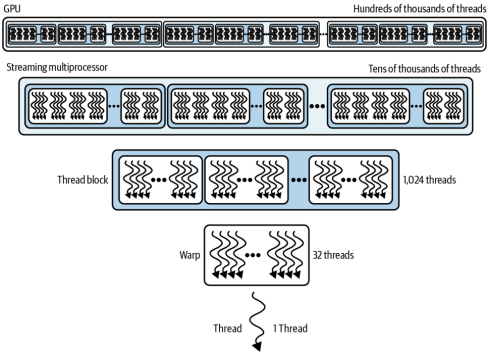

Figure 6-9. Relative scale of thread resources

- Pick block sizes that are **multiples of 32** — a 33-thread block burns two warp slots, one of them 31/32 empty. You hit **per-SM limits** long before grid limits.[1,2]

Anatomy of a CUDA Kernel

```
__global__ void myKernel(float* input, int N) {
    // unique global index for this thread
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) { // bounds check!
        input[idx] *= 2.0f;
    }
}
// host side:
const int threadsPerBlock = 256; // 8 warps
const int blocksPerGrid =
    (N + threadsPerBlock - 1) / threadsPerBlock; // round up
myKernel<<<blocksPerGrid, threadsPerBlock>>>(d_input, N);
cudaDeviceSynchronize();
```

- `__global__` marks device code; `<<<blocks, threads>>>` configures the launch.
- The kernel is written for **one thread**; `idx` carves out its slice of the data.
- The **bounds check** `if (idx < N)` protects the last, partially-filled block from illegal memory accesses.
- Errors surface **lazily** — check `cudaGetLastError()` after launches.

Plain English: write the job for one worker; CUDA prints a million copies, each with a different row number.

Choosing Launch Parameters

Recipe that works almost everywhere

Start with `threadsPerBlock = 256` (8 warps); `blocksPerGrid = (N + 255) / 256` to cover every element. Then tune (128/512) by profiling.

- **Multiple of 32**: no half-empty warps occupying scheduler slots.
- **Latency hiding**: hundreds of threads per SM keep pipelines busy during DRAM stalls.
- **Resource balance**: 256 rarely exhausts registers or shared memory; 256–512 is the sweet spot on Blackwell.
- 2D/3D data: same pattern with `dim3` — e.g., 16×16 blocks over a $1{,}024 \times 1{,}024$ image, with a 2D bounds check.

Plain English: 256 is the "medium coffee" of block sizes — almost never the wrong first order, refine later if profiling says so.

Allocate Asynchronously: Streams and Memory Pools

```
cudaStream_t s;
cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking);

float* d_buf = nullptr;
cudaMallocAsync(&d_buf, N * sizeof(float), s);
myKernel<<<blocks, threads, 0, s>>>(d_buf, N);
cudaFreeAsync(d_buf, s); // deferred until s finishes
```

- Tune the pool: `cudaMemPoolAttrReleaseThreshold`, `cudaMemPoolTrimTo`.

- `cudaMalloc`/`cudaFree` are **synchronous and expensive**: full-device sync + OS calls (`mmap`/`ioctl`).
- `cudaMallocAsync`/`cudaFreeAsync` draw from a per-device **memory pool**: recycled buffers, no global sync, less fragmentation.
- **PyTorch's caching allocator** (`PYTORCH_ALLOC_CONF`) does the same trick for tensors.
- Non-blocking streams avoid legacy **default-stream barriers**.

Plain English: keep a truck fleet in the lot and grab keys — don't buy and return a truck for every delivery.

The GPU Memory Hierarchy (Blackwell)

Level	Scope	Capacity	Latency	BW
Registers	thread	256 KB / SM	∼1 cyc	10s TB/s
Shared + L1	SM	228 KB usable	20–30 cyc	TB/s
TMEM	SM	256 KB (TC)	∼10 cyc	TB/s
Const cache	SM	8 of 64 KB	1 cyc bcast	TB/s
L2 cache	GPU	**126 MB**	∼200 cyc	multi-TB/s
Local (spill)	thread	DRAM-backed	≤1000 cyc	—
Global HBM3e	device	**180 GB** (B200)	≤1000 cyc	**∼8 TB/s**

- Rule: **maximize reuse** in registers/shared/L2; touch HBM as rarely — and as **coalesced** (128-byte aligned) — as possible.

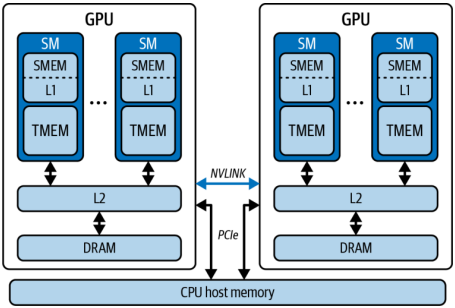

Figure 6-10. GPU memory hierarchy, including the CPU

Plain English: desk (registers) → office shelf (shared) → building library (L2) → warehouse across town (HBM). Do your work at the desk.

TMEM and TMA: Feeding the Tensor Cores

- **TMEM**: dedicated **256 KB per-SM SRAM** that acts as the **accumulator** for 5th-gen Tensor Core ops (`tcgen05.*`, UMMA); tens of TB/s to the Tensor Cores.
- Not pointer-addressable from CUDA C++ — data movement is orchestrated by the **TMA** (Tensor Memory Accelerator) via descriptors: global ↔ SMEM ↔ TMEM.
- For $C = A \times B$: operand B in SMEM, A + accumulator in TMEM; tiles flow from HBM through L2 by TMA.
- Details in Chapter 10 — here, know that it exists and **reduces global-memory pressure**.

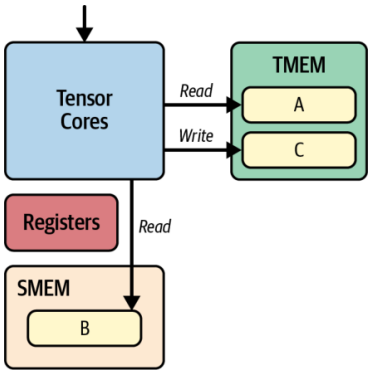

Figure 6-11. TMEM + SMEM servicing Tensor Cores for $C = A \times B$

Plain English: TMA is a forklift crew moving pallets in the background; the chefs never leave the kitchen.

Unified Memory: One Address Space, Hidden Migrations

- `cudaMallocManaged()`: one coherent CPU+GPU address space; pages **migrate on demand** — no manual `cudaMemcpy`.
- Cost: a GPU touch of a CPU-resident page **page-faults and stalls** the kernel.
- On Grace superchips, **NVLink-C2C** (∼900 GB/s) makes migrations far cheaper than PCIe — but never free.
- Fix "surprise" stalls proactively:

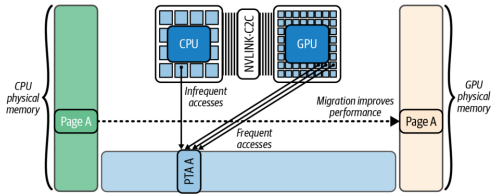

Figure 6-16. Automatic page migration with Unified Memory

```
cudaMemPrefetchAsync(ptr, size, gpuId, s); // move it BEFORE launch
cudaMemAdvise(ptr, size, cudaMemAdviseSetPreferredLocation, gpuId);
cudaMemAdvise(ptr, size, cudaMemAdviseSetReadMostly, gpuId);
cudaStreamAttachMemAsync(stream, ptr, 0, cudaMemAttachSingle);
```

Plain English: Unified Memory is room service: convenient, but if you don't order ahead (prefetch), you wait hungry at kernel time.

Occupancy Case Study: One Thread vs. Many

Sequential — one thread does everything:

```
__global__ void addSequential(const float* A,
    const float* B, float* C, int N) {
  if (blockIdx.x == 0 && threadIdx.x == 0)
    for (int i = 0; i < N; ++i)
      C[i] = A[i] + B[i];
}
addSequential<<<1,1>>>(dA, dB, dC, N);
```

Same trap in PyTorch: a Python `for` loop of `C[i] = A[i] + B[i]` launches **N tiny serial kernels**.

Parallel — one thread per element:

```
__global__ void addParallel(const float* A,
    const float* B, float* C, int N) {
  int idx = blockIdx.x*blockDim.x + threadIdx.x;
  if (idx < N) C[idx] = A[idx] + B[idx];
}
addParallel<<<(N+255)/256, 256>>>(dA, dB, dC, N);
```

PyTorch equivalent: just `C = A + B` — one vectorized kernel.

Plain English: the most common GPU performance bug is not a slow kernel — it's accidentally using the GPU as a very expensive single-core CPU.

Case Study Results: 22× from Occupancy Alone

Metric	Sequential	Parallel
Kernel time (ms)	48.21	**2.17**
GPU utilization	1.5%	**95%**
Achieved occupancy	1.3%	**38.7%**
Warp exec. efficiency	3.1%	**100%**

Table 6-6 (illustrative numbers — see the book's GitHub repo[7] for real benchmarks)

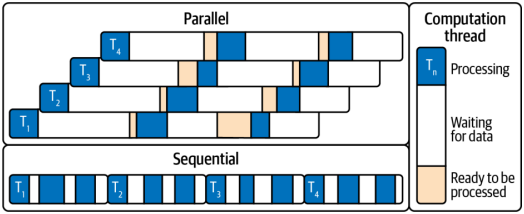

Figure 6-20. Sequential timeline has idle gaps; parallel fills them with other warps' work

- Rule #1 of CUDA performance: **launch enough parallel work**. Aim for occupancy in the **80–100%** range *if* performance is poor and occupancy is low.
- But: at high occupancy with a **memory-bound** kernel, more warps won't help — the bottleneck is bandwidth (see roofline).

Tuning Occupancy: Launch Bounds and the Occupancy API

```
// promise <=256 threads/block; ask for >=16 blocks/SM
__global__ __launch_bounds__(256, 16)
void myKernel(...) { /* ... */ }

// autotune block size at runtime:
int minGridSize = 0, bestBlockSize = 0;
cudaOccupancyMaxPotentialBlockSize(
    &minGridSize, &bestBlockSize, myKernel,
    dynSmemBytes, /*blockSizeLimit=*/0);
int gridSize = std::max(minGridSize,
    (N + bestBlockSize - 1) / bestBlockSize);
```

- `__launch_bounds__` lets the compiler cap registers/inlining so **more warps fit per SM** (hardware clamps impossible requests, e.g. $16 \times 256 > 2{,}048$).
- **Trade-off**: fewer registers per thread $\Rightarrow$ more resident warps, but too few $\Rightarrow$ **spilling to local memory** (slow!).
- **Validate** API suggestions by timing ± 1–2 block-size candidates — slightly sub-maximal occupancy is sometimes faster.

Plain English: you're trading each worker's toolbox size for the number of workers on the floor — the sweet spot is empirical.

Debugging Correctness: NVIDIA Compute Sanitizer

Memory

- **memcheck** — out-of-bounds / misaligned accesses, device leaks.
- **initcheck** — reads of uninitialized global memory (missing H2D copy?).

Concurrency

- **racecheck** — shared-memory hazards (WAW / WAR / RAW).
- **synccheck** — invalid barriers, deadlock-causing sync.

- CLI: `compute-sanitizer [--tool <name>] <app>`; filter with `--kernel-name`, annotate with **NVTX**.
- Put it in **CI** with `--error-exitcode` — catch correctness regressions before they ship.[5]

Plain English: with 100,000 threads, "it works on my run" means nothing — the sanitizer is your seatbelt for race conditions you'll never reproduce by hand.

The Roofline Model: Which Wall Are You Hitting?

- **Arithmetic intensity** = FLOPs per byte moved to/from HBM.
- Two ceilings: **flat** compute roof (∼80 TFLOP/s FP32) and **sloped** memory roof (∼8 TB/s). They cross at the **ridge point** ≈ 10 FLOPs/byte.
- Example: `C = A + B` does 1 FLOP per 12 bytes ⇒ **0.083 FLOPs/byte** — >100× left of the ridge: hopelessly **memory-bound**.
- LLMs: **prefill** tends compute-bound; **decode** (streaming weights) is memory-bound.

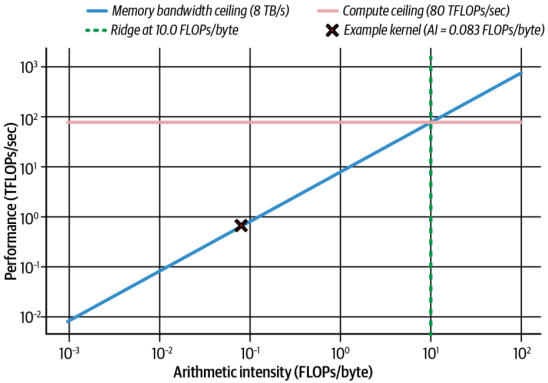

Figure 6-21. Roofline for a Blackwell-class GPU with our kernel at 0.083 FLOPs/byte

Plain English: the roofline asks first: is this kernel starving for math, or for data?[4]

Moving Right on the Roofline: Lower Precision Pays Twice

- To escape the memory-bound region, **do more work per byte**: reuse data on-chip, fuse ops — or simply **shrink the bytes**.
- One 128-byte transaction carries **32 FP32 = 64 FP16 = 128 FP8 = 256 FP4** values. FP32 $\rightarrow$ FP16 instantly **doubles** FLOPs/byte.
- Blackwell adds **FP8 and FP4 Tensor Cores**, plus **hardware decompression**: weights stored compressed in HBM are expanded on the fly — effectively more bandwidth.
- That's why Blackwell shines on memory-bound **token generation**: 4-bit weights in HBM, decompressed and upcast for accumulation.

The takeaway

Precision is a **bandwidth** optimization as much as a compute one: halve the bytes $\Rightarrow$ double the arithmetic intensity $\Rightarrow$ move toward the compute roof.

Profiling Workflow: nsys for *When*, ncu for *Why*

- **Nsight Systems** (`nsys`): whole-app timeline — GPU idle gaps, CPU overlap, PCIe/NVLink transfers. *Plain English: a stopwatch for the whole machine.*
- **Nsight Compute** (`ncu`): per-kernel counters — occupancy, stall reasons, cache hit rates, DRAM utilization. *Plain English: a microscope on one kernel.*
- Wrap suspect regions in **NVTX** ranges to see them on the timeline.
- Memory-bound signature: **high DRAM utilization + low ALU utilization**; missing async overlap often = **no pinned memory** or default-stream sync.

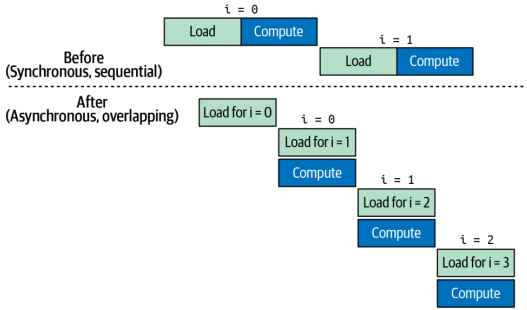

Figure 6-22. Sync (sequential) vs. async (overlapped) transfers + compute

Key Takeaways

- **SIMT**: 32-thread warps in lockstep; keep many warps in flight to hide latency.
- **Hierarchy**: threads → blocks ($\leq$1,024) → grids; blocks independent ⇒ portable scaling.
- **Block size**: multiples of 32; start at 256; respect per-SM limits (64 warps, 32 blocks, 228 KB shared, 255 regs/thread).
- **Async allocation**: `cudaMallocAsync` + streams + pools; PyTorch's caching allocator does this for you.
- **Memory ladder**: registers → shared/L1 → L2 → HBM; reuse up high, coalesce down low.
- **Unified Memory**: convenient, but prefetch + advise to avoid page-fault stalls.
- **Roofline**: FLOPs/byte decides memory- vs. compute-bound; lower precision moves you right.
- **Occupancy is necessary, not sufficient**: high occupancy can't fix a bandwidth-limited kernel — profile, then optimize.

Conclusion and What's Next

This chapter in one sentence

Make the GPU **busy** (occupancy), make data **close** (memory hierarchy), and let the **roofline** tell you which battle to fight next.

- Max occupancy is **not always optimal**: enough ILP per thread can beat more warps; sometimes fewer threads with more registers each wins. **Benchmark it.**
- Coming next in the book: warp divergence and memory-access tuning (Ch. 7–8), arithmetic intensity (Ch. 9), TMA and warp specialization (Ch. 10).
- My second talk — **Chapter 12** — builds on this: once single kernels are fast, how do we **orchestrate** them so the GPU never waits for the CPU?

Questions?

References & Further Reading

1. C. Fregly. *AI Systems Performance Engineering* (O'Reilly, 2026), Chapter 6.
2. NVIDIA. *Blackwell Tuning Guide* and *Blackwell Architecture Whitepaper* — SM limits, dual-issue schedulers, TMEM/tcgen05.
3. NVIDIA. *CUDA C++ Programming Guide* — thread hierarchy, Unified Memory, occupancy API, `__launch_bounds__`, stream-ordered allocation.
4. S. Williams, A. Waterman, D. Patterson. *Roofline: An Insightful Visual Performance Model for Multicore Architectures*. CACM 52(4), 2009.
5. NVIDIA. *Compute Sanitizer User Guide* — memcheck, racecheck, initcheck, synccheck.
6. NVIDIA. *Nsight Systems* and *Nsight Compute* documentation — `nsys profile`, `ncu --section SpeedOfLight`, NVTX.
7. Book code and benchmarks: the book's GitHub repository (per-architecture results for the illustrative tables in this deck).